Aikyam Ananda Pre-School

Comprehensive Interview Questions Guide

1. Technical Developer Interview Questions

These questions evaluate a developer's knowledge in maintaining, optimizing, and securing this codebase. Topics include Firebase Realtime Database, ES Modules, performance profiling, responsive CSS architecture, and keyboard accessibility.

Q1: Client-Side Security & Firebase Realtime Database Config

The Firebase configuration credentials (like apiKey, databaseURL, etc.) are hardcoded directly in script.js on the client side. Is this a security risk, and how would you protect the Firebase Realtime Database from unauthorized read/write requests?

Expected Answer: Firebase client-side configuration keys are not secrets (they are required for the client browser to connect to Firebase services). However, without proper **Firebase Security Rules**, anyone can read/write/delete database entries since the database URL is public. To secure the database, we must configure rules in the Firebase Console. For example, we should only allow write access when specific schemas are validated, and restrict read access to authenticated administrators only.

```
{
  "rules": {
    "enquiries": {
      ".read": "auth != null",
      ".write": "newData.hasChildren(['parentName', 'childName', 'dob', 'contactNo', 'email'])
"
    },
    "feedback": {
      ".read": "auth != null",
      ".write": "newData.hasChildren(['name', 'email', 'message'])"
    }
  }
}
```

Q2: Error Handling in Chained Async Submissions

When an enquiry is submitted, the form pushes data to Firebase RTDB and then sends an email notification via EmailJS. If Firebase succeeds but EmailJS fails (e.g. quota limit), how does the website currently behave, and how would you improve this?

Expected Answer: Currently, the EmailJS logic is nested inside a separate try-catch inside the main form submit success block. If EmailJS fails, the error is caught, logged to the console, and the user is still shown a successful alert: *'Thank you for your enquiry!'*. To improve this resilience and secure API templates, we could move EmailJS to a serverless function or a Firebase Cloud Function that triggers automatically on a new entry write to the database.

Q3: Child Age Calculation Logic

The enquiry form calculates the child's age dynamically as of a static date: July 31, 2026. Explain the logic used in script.js to handle day borrows (negative differences in days) and month borrows when calculating years, months, and days.

Expected Answer: The calculation subtracts years, months, and days. If the day difference is negative, it decrements the month difference and borrows the number of days in the previous month using:
const prevMonth = new Date(targetDate.getFullYear(), targetDate.getMonth(), 0);
days += prevMonth.getDate();
If the month difference goes negative, it decrements the year and adds 12 to the months. This handles date boundaries manually without needing external libraries.

Q4: Lightbox Event Delegation

The gallery page attaches a click event listener to each image inside the grid using a .forEach loop. What is the performance bottleneck if this grid scales to hundreds of images, and how would you optimize it using event delegation?

Expected Answer: Attaching individual event listeners to hundreds of images increases memory usage and reduces page responsiveness. We can optimize it by attaching a single click event listener to the parent .gallery-grid container and checking the e.target tag name:

```
const grid = document.querySelector('.gallery-grid');
grid.addEventListener('click', (e) => {
    if (e.target.tagName === 'IMG') {
        const index = Array.from(grid.querySelectorAll('img')).indexOf(e.target);
        lightbox.style.display = 'flex';
        showImage(index);
    }
});
```

Q5: Events Carousel Loops & Hovers

How does the events carousel handle loop resets when it reaches the end of the scrollable items, and how is the auto-scroll interval managed during user interaction?

Expected Answer: The carousel compares scrollLeft with maxScroll - 10. If it exceeds that threshold, it scrolls back to 0. The auto-scroll is run via an interval that is cleared when the mouse enters (mouseenter) and reset when the mouse leaves (mouseleave) the carousel wrapper.

Q6: Mobile Header & Hamburger Menu

How does the mobile navigation toggle work? Explain how the CSS and JS classes interact.

Expected Answer: In script.js, clicking the .hamburger icon toggles the .active class on both the hamburger and .nav-links container. In CSS, .nav-links has rules configured (such as absolute positioning) that display and slide the menu in when the .active class is present.

Q7: Wavy Section Dividers (SVG Masking)

How are the wavy boundaries between sections styled and positioned relative to the content sections? Also discuss how the footer and header are shaped.

Expected Answer: The waves between sections are inline SVG elements placed inside `.wave-top` and `.wave-bottom` divs. These are positioned absolutely (`position: absolute`) at the top and bottom edge of their containers (`width: 100%`, `height: 50px`). The header and footer, on the other hand, utilize CSS mask-image SVGs (`-webkit-mask-image: url(...)`) to clip their bottom and top edges into wavy shapes respectively.

Q8: Modular JS vs Classic Scripts

The script is imported using type="module" in the HTML templates. What are the key characteristics of ES Modules compared to classic scripts, and how does this affect global variables?

Expected Answer: ES Modules: 1. Always execute in strict mode. 2. Are deferred by default (loaded after HTML parsing, preserving execution order). 3. Keep variables scoped locally to the file module rather than creating global variables on the window object. This is why we can safely import Firebase SDK components (`import { initializeApp } from ...`) inside `script.js` without polluting the global namespace.

Q9: CSS Custom Properties (Variables) & Theming

Explain how CSS custom properties (variables) are defined in style.css. If you were asked to implement a dark theme toggle, how would you leverage these variables?

Expected Answer: Custom properties are defined in the :root selector in style.css (e.g., --primary-bg: #00BFFF; --text-color: #333333;). To implement a dark theme, we would override these variable values under a body selector class (e.g. body.dark-theme { --primary-bg: #121212; --text-color: #f5f5f5; }) and toggle that class using Javascript. This updates all styles using var(--primary-bg) automatically.

Q10: Scroll-Linked Animations (Intersection Observer)

The website triggers slide-in animations for section titles. Why is the IntersectionObserver API used instead of listening to the window scroll event?

Expected Answer: Traditional window scroll listeners run on the main thread every time the user scrolls, creating scroll lag and performance issues due to heavy layout calculations. IntersectionObserver runs asynchronously and notifies the browser only when the observed target (like .animated-title) intersects the viewport, which is highly optimized and prevents layout thrashing.

Q11: Accessibility & Keyboard Events in Custom Lightbox

The lightbox implementation listens to Escape, ArrowLeft, and ArrowRight key events. How is this event handler registered, and what are the best practices to prevent key events from firing when the lightbox is closed?

Expected Answer: The keyboard event listener is registered globally on the document object: document.addEventListener('keydown', (e) => { ... }). To avoid performance degradation or side-effects when the lightbox is closed, the handler wraps the key checks inside an active visibility condition (e.g., checking if `lightbox.style.display === 'flex'`). Best practices include using `aria-hidden="true/false"` and implementing focus trapping so keyboard navigation stays within the active lightbox.

Q12: EmailJS Template Parameters & Input Sanitization

When submitting the enquiry form, form data is compiled into templateParams and sent to EmailJS. Since user input goes directly into an email, what security measures would you implement to prevent email header injection or spam scripting?

Expected Answer: Because EmailJS templates render variables safely as text nodes, standard HTML injection in the email body is neutralized. However, to prevent spam scripts, header injection, or automated bot submission abuse, we should implement frontend and backend checks: 1. Honeypot input fields to catch bots. 2. Regular expression validation for email addresses. 3. Input length constraints. 4. String sanitization to strip script and HTML tags.

2. Parent & Admission Interview Questions

These questions are designed for the school administration to ask parents during admission interviews, aligning with the data captured on the website forms and the school's developmental philosophy.

Q1: Pre-School Exposure & Age Verification

I see from your application that [Child's Name] will be [Age] on July 31, 2026. Have they previously attended a playgroup, daycare, or nursery? How was their experience there?

Purpose: Verifies the age calculated by the form and establishes the child's baseline familiarity with structured classroom settings.

Q2: Transportation & Commute Logistics

You mentioned you live in [Area of Residence]. How do you plan to manage the commute? Are you interested in school transport options or carpooling?

Purpose: Assesses logical feasibility of attendance and helps structure school transport or bus route allocations.

Q3: Alignment with Holistic Development Focus

Our curriculum focuses on emotional, cognitive, physical, and social skill development. Which of these areas do you think your child is strongest in, and where do they need the most support?

Purpose: Checks if parents understand the school's focus and helps teachers prepare individualized care for the child.

Q4: Collaborative Learning & Separation Anxiety

How does your child react to new social environments? What comfort strategies do you use when they face separation anxiety?

Purpose: Helps school counselors and teachers manage the child's transition during the critical first few weeks of school.

Q5: Campus Visit & Open House Interests

You indicated interest in visiting the campus for an Open House. What are the key elements you look for in a preschool classroom and playground?

Purpose: Tailors the campus tour to parent priorities (e.g. hygiene, safety features, toy variety, teacher-student ratios).